

c program to Extract a group of bits



A common C interview question is: **"Extract n bits starting at position p from an integer."**

1. Basic formula

Suppose:

```
x = 1101101011
      ↑↑↑
      extract 3 bits starting at position 3
```



Assume positions are counted from the **right, starting at 0**:

```

      9 8 7 6 5 4 3 2 1 0
x =   1 1 0 1 1 0 1 0 1 1
              ↑ ↑ ↑
              3 2 1
```



To extract n bits starting at position p :

```
</> C
```



2. C program

```
#include <stdio.h>
#include <stdint.h>

uint32_t extract_bits(uint32_t x, int p, int n)
{
    uint32_t mask = (1U << n) - 1;
    return (x >> p) & mask;
}
```

```
int main(void)
{
    uint32_t x = 0x6DB; // 11011011011

    int p = 3;
    int n = 4;

    uint32_t result = extract_bits(x, p, n);

    printf("Result = %u\n", result);
    printf("Result = 0x%X\n", result);
}
```

</> C



For:

```
x = 0x6DB
p = 3
n = 4
```



The operation is:

```
x >> 3
```



which moves the desired bits to the lowest positions.

Then:

```
(1U << 4) - 1
= 10000 - 1
= 01111
```



So:

```
result = (x >> 3) & 0xF;
```



3. Why the mask works

For extracting **4 bits**:

</> C



gives:

```
0001 0000
    -
0000 0001
-----
0000 1111
```



So the mask keeps only 4 bits.

For $n = 3$:

```
</> C
```



gives:

```
00000111
```



For $n = 8$:

```
11111111
```



Interview-ready function

```
uint32_t extract_bits(uint32_t v, int n) {
    </> C
}
```

✦ Upgrade



Pattern to remember:

```

      SHIFT      MASK
        ↓         ↓
(x >> position) & ((1 << number_of_bits) - 1)
```



One important edge case: if $n == 32$, $1U << n$ is invalid/undefined because you're shifting a 32-bit value by 32. For a robust interview implementation, handle that case separately.



How to handle it

Yes — the important edge case is `n == 32`. You cannot do:

```
</> C
```



```
1U << 32
```

for a 32-bit `unsigned int`; shifting by the width of the type is undefined behavior.

Robust implementation

```
</> C
```



```
#include <stdio.h>
#include <stdint.h>

uint32_t extract_bits(uint32_t x, int p, int n)
{
    uint32_t mask;

    if (n == 32)
        mask = UINT32_MAX;
    else
        mask = (1U << n) - 1;

    return (x >> p) & mask;
}

int main(void)
{
    uint32_t x = 0x12345678;

    printf("0x%X\n", extract_bits(x, 0, 32));

    return 0;
}
```

Ask ChatGPT

Share highlighted

But there is another edge case

If you're extracting n bits starting at position p , you must ensure:

$p + n \leq 32$



Otherwise you're asking for bits that don't exist.

A more interview-ready version is:

`</> C`



```
uint32_t extract_bits(uint32_t x, int p, int n)
{
    if (p < 0 || n <= 0 || p >= 32 || p + n > 32)
        return 0;

    uint32_t mask;

    if (n == 32)
        mask = UINT32_MAX;
    else
        mask = (1U << n) - 1;

    return (x >> p) & mask;
}
```

One subtle point

With this definition:

p = starting bit position from LSB
 n = number of bits



$n == 32$ is only valid when:

$p == 0$



because:

$p + n \leq 32$



So the cleanest version is:

`</> C`



```
uint32_t extract_bits(uint32_t x, int p, int n)
{
    if (p < 0 || n <= 0 || p + n > 32)
        return 0;

    if (n == 32)
        return x;

    return (x >> p) & ((1U << n) - 1);
}
```

Interview tip: When you see $(1U \ll n) - 1$, immediately think about the $n == 32$ **shift-width edge case**. That's exactly the kind of thing an interviewer may ask as a follow-up.



ChatGPT can make mistakes. Check important info.